Lab4.1: (EXP 7)

2. Error Handling and Unit Testing

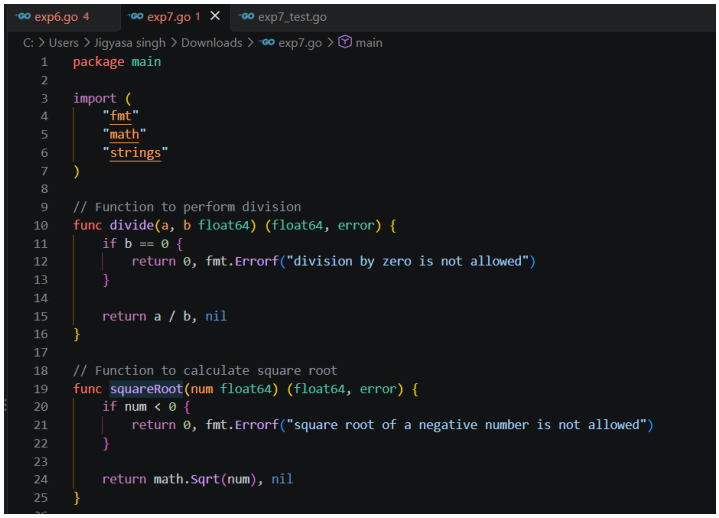
Write a Go program that:

(1) handles at least three distinct error scenarios using the error type (e.g., division by zero, negative square root, empty input) with descriptive messages using fmt.Errorf; and

(2) includes a _test.go file with at least three test functions using the testing package, covering valid inputs and each error case.

• Run go test -v and capture the result.

• Push all source and test files to your GitHub repository. Upload screenshots of go test -v output.





```go
func main() {
    fmt.Println("1. Division Error:")
    result, err := divide(10, 0)
    if err != nil {
        fmt.Println("Error:", err)
    } else {
        fmt.Println("Result:", result)
    }

    fmt.Println("\n2. Square Root Error:")
    result, err = squareRoot(-25)
    if err != nil {
        fmt.Println("Error:", err)
    } else {
        fmt.Println("Result:", result)
    }

    fmt.Println("\n3. Empty Input Error:")
    err = validateInput("")
    if err != nil {
        fmt.Println("Error:", err)
    } else {
        fmt.Println("Input is valid")
    }

    fmt.Println("\n4. Valid Operations:")
    result, err = divide(20, 5)
    if err == nil {
        fmt.Println("20 / 5 =", result)
    }

    result, err = squareRoot(64)
    if err == nil {
        fmt.Println("Square root of 64 =", result)
    }
```

```go
// Test square root function
run test | debug test
func TestSquareRoot(t *testing.T) {
    // Valid input
    result, err := squareRoot(64)

    if err != nil {
        t.Errorf("Unexpected error: %v", err)
    }

    if result != 8 {
        t.Errorf("Expected 8, got %f", result)
    }

    // Error case: negative number
    _, err = squareRoot(-25)

    if err == nil {
        t.Error("Expected negative square root error")
    }

    if !strings.Contains(err.Error(), "negative number") {
        t.Errorf("Unexpected error message: %v", err)
    }
}

// Test input validation function
run test | debug test
func TestValidateInput(t *testing.T) {
    // Valid input
    err := validateInput("Hello Go")

    if err != nil {
        t.Errorf("Unexpected error: %v", err)
    }
```

exp6.go 4 exp7.go 1 exp7_test.go

C: > Users > Jigyasa singh > Downloads > exp7_test.go > TestSquareRoot

run package tests | run file tests

```go
package main

import (
    "strings"
    "testing"
)

// Test division function
run test | debug test
func TestDivide(t *testing.T) {
    // Valid input
    result, err := divide(20, 5)

    if err != nil {
        t.Errorf("Unexpected error: %v", err)
    }

    if result != 4 {
        t.Errorf("Expected 4, got %f", result)
    }

    // Error case: division by zero
    _, err = divide(10, 0)

    if err == nil {
        t.Error("Expected division by zero error")
    }

    if !strings.Contains(err.Error(), "division by zero") {
        t.Errorf("Unexpected error message: %v", err)
    }
}

// Test square root function
run test | debug test
func TestSquareRoot(t *testing.T) {
```

```go
// Test input validation function
run test | debug test
func TestValidateInput(t *testing.T) {
    // Valid input
    err := validateInput("Hello Go")

    if err != nil {
        t.Errorf("Unexpected error: %v", err)
    }

    // Error case: empty input
    err = validateInput("")

    if err == nil {
        t.Error("Expected empty input error")
    }

    if !strings.Contains(err.Error(), "empty") {
        t.Errorf("Unexpected error message: %v", err)
    }
}
```

```
PS C:\Users\Jigyasa singh\Downloads\exp7> go run "c:\Users\Jigyasa singh\Downloads\exp7.go"
1. Division Error:
Error: division by zero is not allowed

2. Square Root Error:
Error: square root of a negative number is not allowed

3. Empty Input Error:
Error: input cannot be empty

4. Valid Operations:
20 / 5 = 4
Square root of 64 = 8
Input is valid
PS C:\Users\Jigyasa singh\Downloads\exp7> go test -v
=== RUN   TestDivide
--- PASS: TestDivide (0.00s)
=== RUN   TestSquareRoot
--- PASS: TestSquareRoot (0.00s)
=== RUN   TestValidateInput
--- PASS: TestValidateInput (0.00s)
PASS
ok      experiment7/exp7        0.475s
```